-

Graph Neural Networks

DR SRinivasa L.Chakravarthy
chakri.ls@wilp.bits-pilani.ac.in

AIML ZG514: Graph Neural Networks

BITS Pilani
July 10, 2026

Outline of presentation

1. Course Information
2. Why Graphs?
3. Why Graph Neural Networks (GNNs)?
4. Basics of Graph Theory

2 Course Information

Course Outline

We'll cover **machine learning and deep learning for graph data**.

- Basics of graph theory
- Graph data and representations
- Node embeddings
- Graph representation learning
- Message passing neural networks
- The Weisfeiler–Lehman algorithm and expressive power
- Deep generative models for graphs
- Knowledge graph embeddings
- Applications
- Research frontier

Prerequisites

The course is **self-contained**, but some prior background will be helpful.

Mathematical background:

- Machine learning
- Algorithms and graph theory
- Probability and statistics
- Linear algebra

Programming: Should be able to write non-trivial programs in Python. PyTorch will be covered in this course.

Readings

- **Graph Representation Learning** by William L. Hamilton.
  `https://www.cs.mcgill.ca/~wlh/grl_book/`

- **A Gentle Introduction to Graph Neural Networks** (Distill).
  `https://distill.pub/2021/gnn-intro/`

- **Research papers** (distributed throughout the course).

Grading Policy

Component	Details	Weight
EC1	2 quizzes (7.5% each) + 2 assignments (7.5% each)	30%
EC2 (Midterm)	2 hours, **closed book**	30%
EC3 (Final exam)	3 hours, **open book**, cumulative (all 16 sessions)	40%
Total		**100%**

- **EC1** consists of two quizzes and two assignments, each worth 7.5%.
- **EC3** is cumulative and covers material from all 16 sessions.

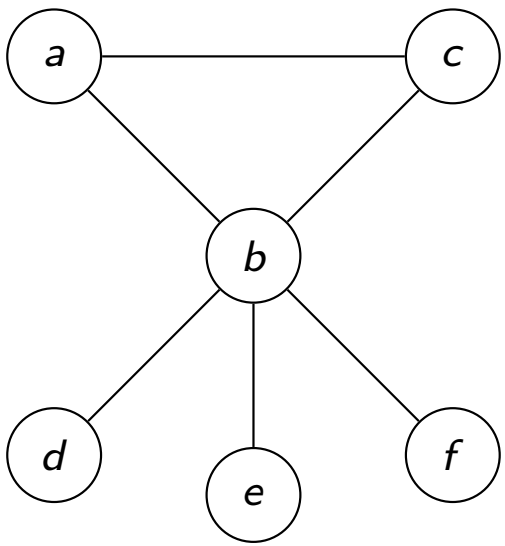
3 Why Graphs?

What is a graph?

Definition: A graph (aka undirected graph) is an ordered pair $G = (V, E)$ consisting of a set V of vertices and a set E of edges, where E is a set of unordered pairs of elements from V.

Example: Let $G = (V, E)$, where
$V = \{a, b, c, d, e, f\}$, and
$E = \{\{a, b\}, \{a, c\}, \{b, c\}, \{b, d\}, \{b, e\}, \{b, f\}\}$.

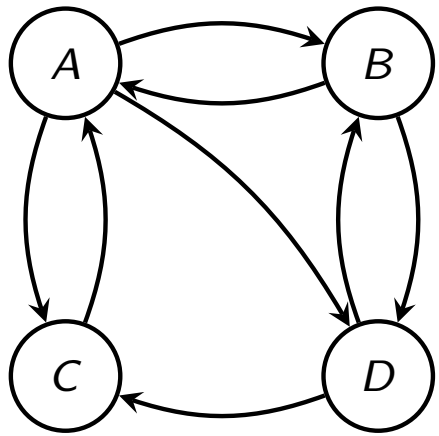
A graph can be represented by a drawing. Two vertices x, y are adjacent if $\{x, y\}$ is an edge, and we say x is a neighbor of y, denoted $x \sim y$. The degree of a vertex x is the number of neighbors of x.



Directed graphs

Definition: A digraph (aka directed graph) is an ordered pair $G = (V, E)$ consisting of a set V of vertices and a set E of edges (aka arcs), where E is a set of *ordered* pairs of elements from V.

Example: Define the digraph $G = (V, E)$ by $V = \{A, B, C, D\}$, $E = \{(A, B), (A, C), (A, D), (B, A), (B, D), (C, A), (D, B), (D, C)\}$.
For example, $V =$ set of all URL's of webpages, and $E = \{(x, y) :$ webpage x has a link to webpage $y\}$.



Graphs: A General Language for Relational Data

Graphs describe **entities** and the **relations/interactions** between them. **Examples across domains:**

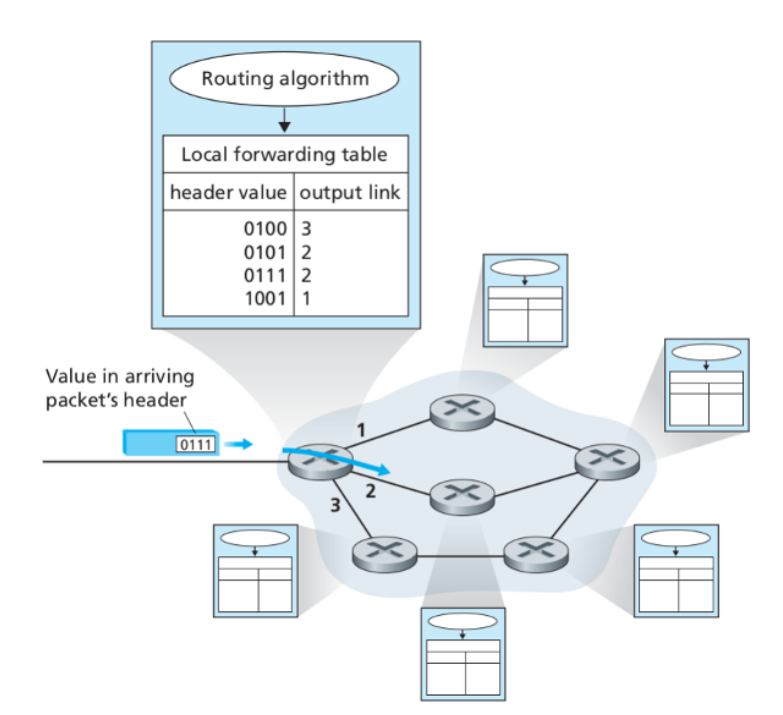
Domain	Vertices	Edges
Social network	People	Friendships
Citation network	Papers	Citations
Molecule	Atoms	Chemical bonds
Web	Webpages	Hyperlinks
Knowledge graph	Entities	Relationships
Transaction graph	Accounts	Transactions

Graph data: Many real-world problems are naturally represented as graphs. GNNs are designed to learn directly from this structure.

Computer Networks

Define the graph $G = (V, E)$:

- $V :=$ set of hosts, computers, or routers
- E: $x, y \in V$ are adjacent whenever there is a direct communication link between x and y



Nodes in the internet execute **shortest path routing algorithms** to determine the next neighbor to forward a packet to.

(Figure: KuroseRoss)

Road Networks

Define the graph $G = (V, E)$:

- $V :=$ set of intersections of roads
- E: $x, y \in V$ are adjacent whenever they are joined directly by a road

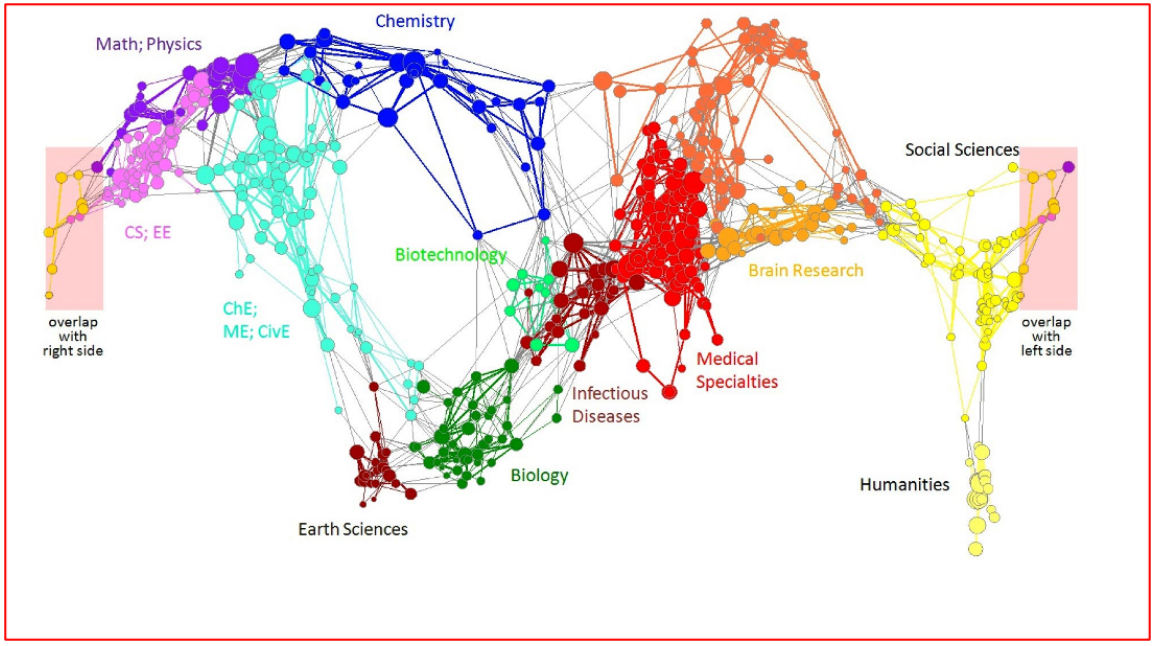
Google Maps runs algorithms on graphs to compute shortest routes, saving time to drive to work. Edges can have **weights** (the distance or time-delay for traveling on that edge).

(Figure: mathsection.com)

Citation Networks

Define the graph $G = (V, E)$:

- $V :=$ set of papers published in journals or conferences
- $(x, y) \in E$ if paper x cites paper y



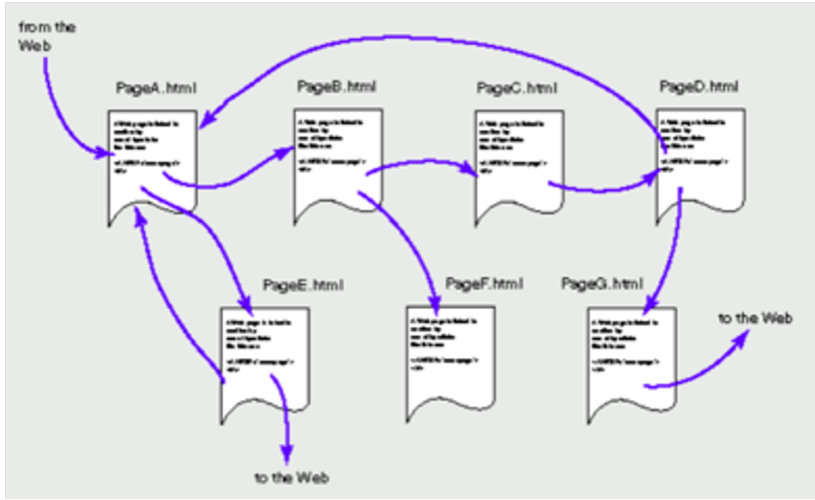
Note: edges are **directed** — citation is a one-way relationship.

(Figure: mmds.org)

The Web Graph

Define the graph $G = (V, E)$:

- $V :=$ set of URL's of web pages
- E: (x, y) is an arc whenever webpage x has a hyperlink to webpage y.



The "links" are hyperlinks (created using the anchor tag in HTML), not physical links.

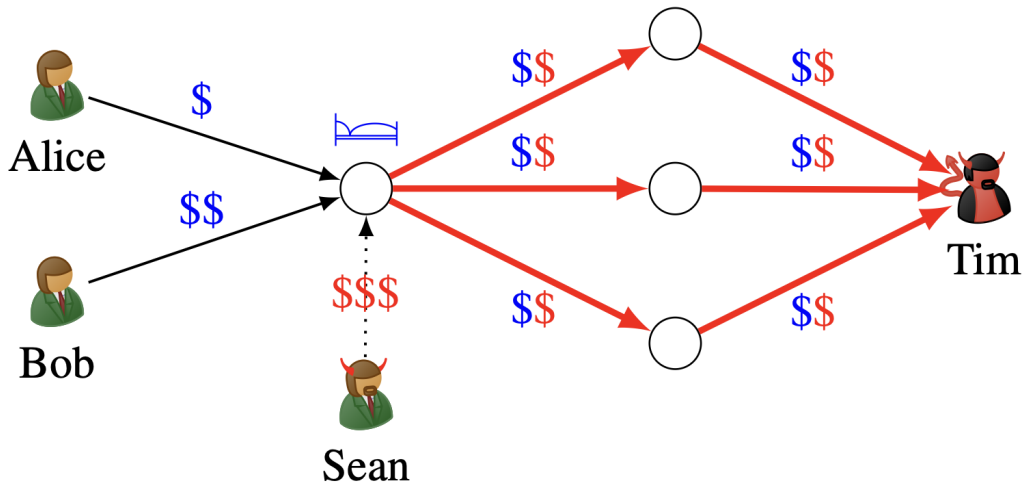
(Figure: ccsu.edu)

Google's PageRank algorithm ranks webpages by the stationary distribution of a random walk on the web graph — equivalently, the principal eigenvector (eigenvalue 11 1) of the Google matrix, a column-stochastic version of the adjacency matrix with teleportation.

Financial Transaction Graphs

Define the graph $G = (V, E)$:

- $V :=$ set of accounts (individuals, companies)
- $(x, y) \in E$ if account x transfers money to account y



Edges carry **features**: amount and timestamp. The graph is **directed** (money flows one way) and a **multigraph** (multiple transactions between the same pair).

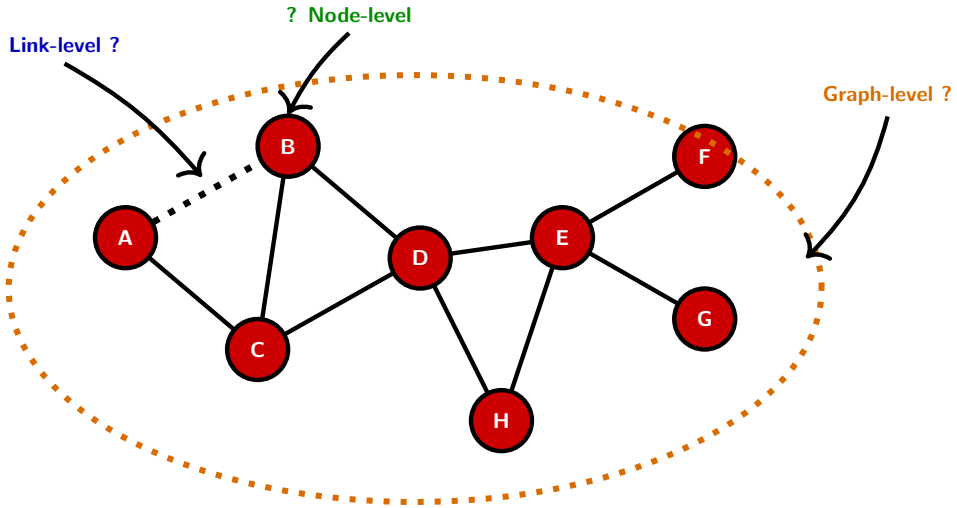
Example: a money-laundering scheme. Sean (criminal) wants to transfer dirty money to his accomplice Tim without raising flags. He funnels it through a front-business hotel, which also receives legitimate payments from Alice and Bob; the hotel then pays shell companies (food, laundry, consulting) that each forward money on to Tim.

(Figure: Egressy et al., NeurIPS 2024.)

4 Why Graph Neural Networks (GNNs)?

Types of ML prediction tasks on graph data

The ML tasks you already know—*classification* and *regression*—show up in three forms on graph data, one per structural level of the graph:



Node-level — predict a label/value *per node*.

- Topic of a paper in a citation network
- User category in a social network
- Protein function in a PPI network

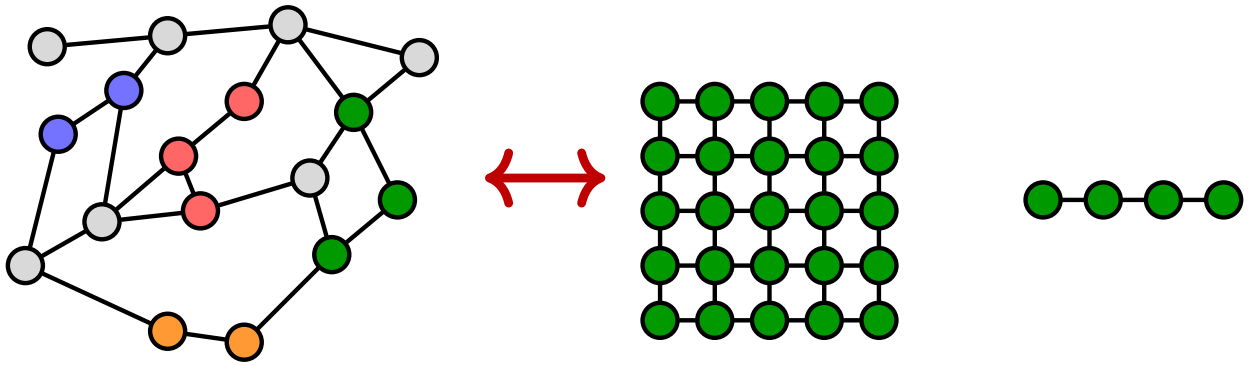
Link-level — predict whether / how *two nodes relate*.

- Friend or product recommendation
- Drug–drug interaction
- Missing facts in a knowledge graph

Graph-level — predict a label/value *for the whole graph*.

- Molecule toxicity / solubility
- Protein fold classification
- *De novo* molecule generation

Why graphs need a new toolkit



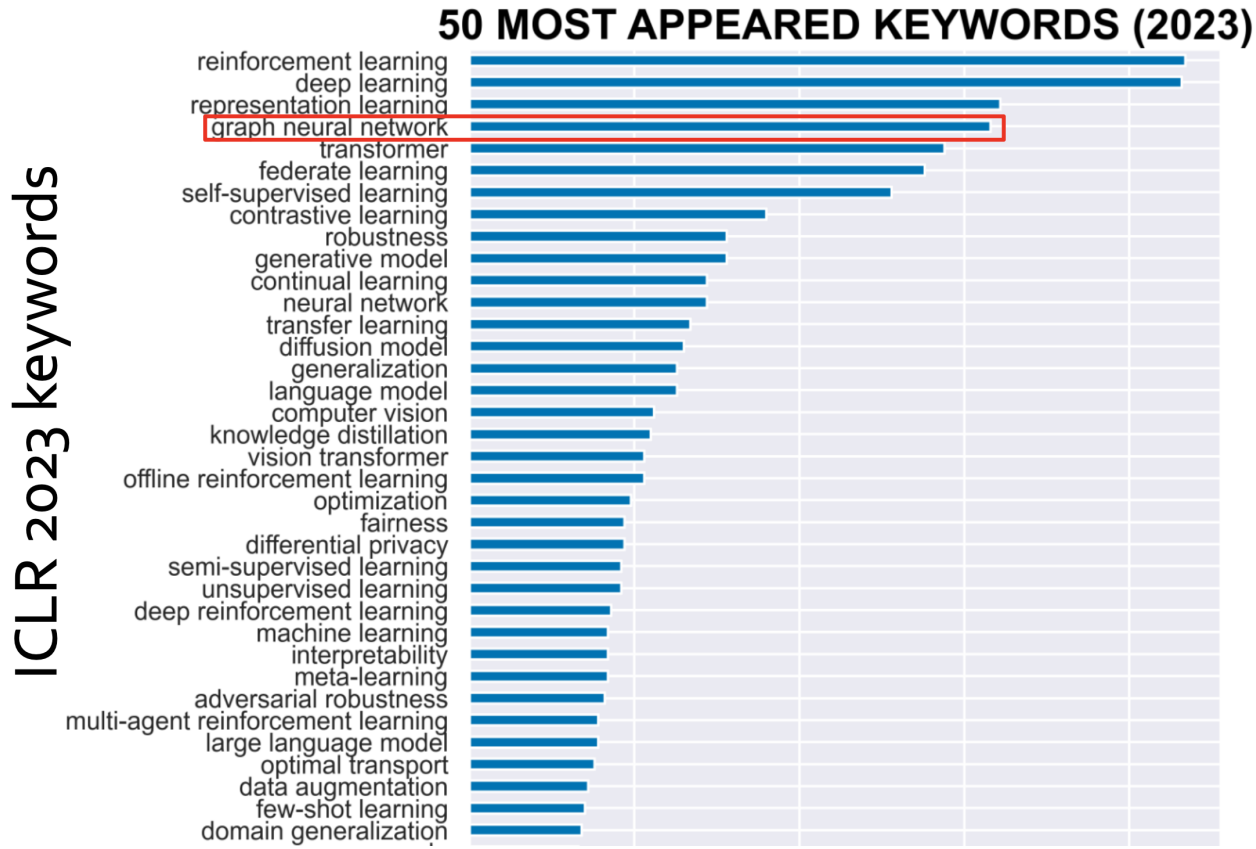
- Images and text: **regular structure**—fixed number of neighbors, canonical ordering, Euclidean geometry.
- Graphs: **none** of these.

We have powerful architectures for grids (**CNNs**) and sequences (**Transformers**).

What is the analogous architecture for graphs?

This course answers: the **Graph Neural Network**.

GNNs are a hot subfield of ML



At ICLR 2023, *graph neural network* was the **4th most frequent keyword**, behind only reinforcement learning, deep learning, and representation learning.

(Figure from Leskovec, Stanford CS224W.)

Why do we need GNNs?

Many real-world problems are naturally represented as graphs: molecules, social networks, road networks, citation graphs, protein contact maps, crystal lattices.

But a deeper question remains:

Given that the data is graph-structured, why do we need a new toolkit?

Why not just apply a CNN, an RNN, or a Transformer, the way we would for images, audio, or text?

Plan for this section:

- Why standard architectures fail on graphs, and which symmetries a graph-aware architecture must respect.
- Six flagship applications of GNNs—weather, maps, recommendation, materials, drugs, chips.

Three properties of graphs that break CNNs, RNNs, Transformers

CNNs, RNNs, and Transformers work because they exploit rigid geometric structure: a 2D grid of pixels (each with 8 canonical neighbors) or a 1D sequence of tokens (each with a canonical predecessor and successor).

Attributed graphs $G = (V, E, w, h)$ have **none** of these properties:

1. Variable neighborhood size. A node may have any number of neighbors—millions for a social-media star, zero for an isolated node. A fixed-size kernel does not apply.
2. No canonical ordering of neighbors. A pixel has a distinguished *north*, *east*, etc. neighbor; the three neighbors of a carbon atom do not. Any architecture that indexes neighbors must pick an arbitrary order or pay a factorial cost.
3. Non-Euclidean geometry. Grids and sequences live in $\mathbb{R}^d$ with Euclidean distance. Graphs, in general, are not embedded in any Euclidean space—the intrinsic metric is shortest-path distance.

Two symmetries a GNN must respect

Any architecture for graphs must respect *two* symmetries:

1. **Permutation invariance** (at the aggregation step):

   Pooling a node's neighbors into a single vector must depend only on the *multiset* of neighbor features, **not on their ordering**.

2. **Permutation equivariance** (at the layer level):

   Relabeling the vertices of the graph must produce a *corresponding* relabeling of the output features—their *values* stay the same.

> A **Graph Neural Network** is a neural architecture that enforces *both* symmetries by construction, via *message passing*.

Why the "right" inductive bias matters

Objection: a large enough MLP on a flattened adjacency matrix can, in principle, approximate any function of a graph.

Response: true in principle (universal approximation), misleading in practice.

- An MLP would have to *learn* permutation invariance from data.
- That requires training data covering *every permutation* of every input graph—combinatorially infeasible.
- A GNN generalizes from *exponentially less data*, because the symmetry is built in.

GNNs are to graphs what CNNs are to images.

A CNN outperforms an MLP on images not because the MLP *can't* represent translation-invariant functions, but because the CNN doesn't have to *learn* translation invariance—it can spend its parameters learning the task.

Flagship applications: six case studies

GNNs have produced state-of-the-art or production-grade results across a wide range of scientific and industrial domains. Each of the next six slides follows the same four-part template:

(i) the **problem**, (ii) the **graph formulation**,
(iii) the **role of the GNN**, (iv) the **headline empirical result**.

Domain	**Flagship**
Atmospheric science	GraphCast (DeepMind)
Transportation	Google Maps ETA
Web-scale recommendation	PinSage (Pinterest)
Materials chemistry	GNoME (DeepMind)
Medicinal chemistry	Halicin (MIT)
Hardware design	AlphaChip (Google)

GraphCast: medium-range global weather forecasting

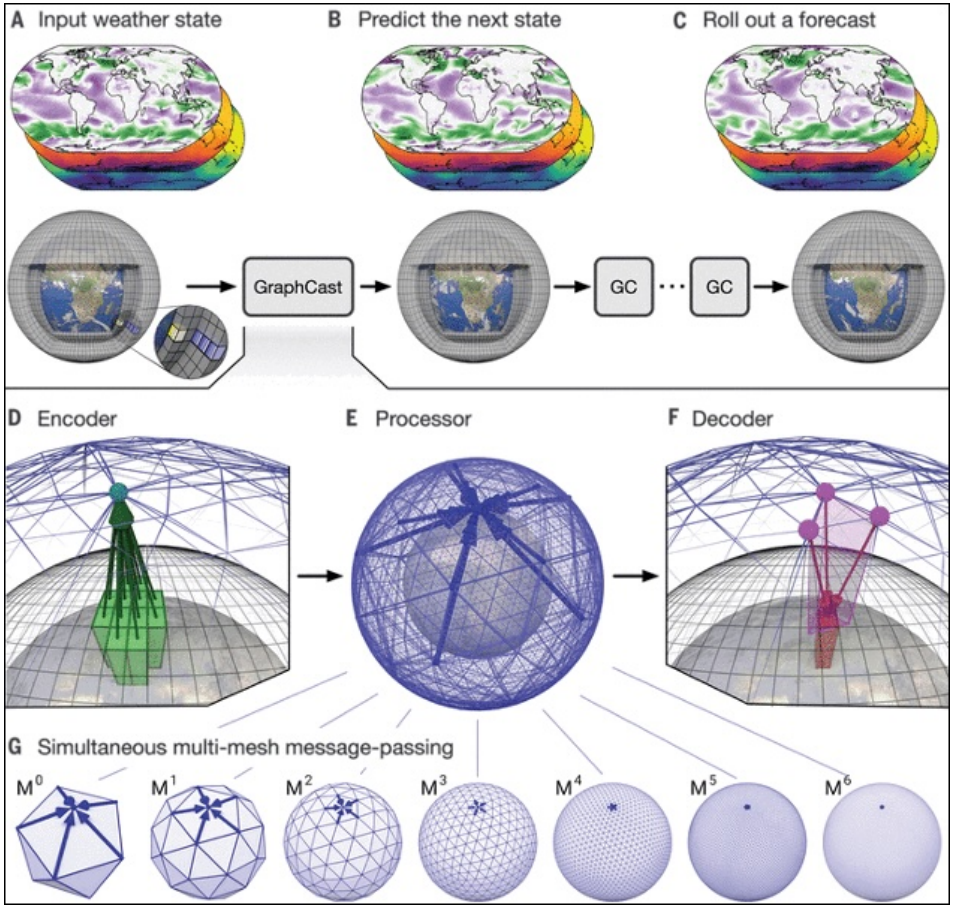
Problem: predict the global atmospheric state up to 10 days ahead. For 50+ years, done by numerically solving PDEs on a supercomputer (ECMWF's HRES).

Graph: multi-scale icosahedral mesh on the globe; vertices are points on the sphere; edges connect close points at multiple resolutions.

GNN role: learned message passing replaces the PDE solver.

Headline: beats HRES on $\sim$90% of 1,380 targets; full 10-day forecast in <1 min on a single TPU. Predicted Hurricane Lee's Nova Scotia landfall 3 days earlier than operational systems.

(Lam et al., *Science* 2023.)

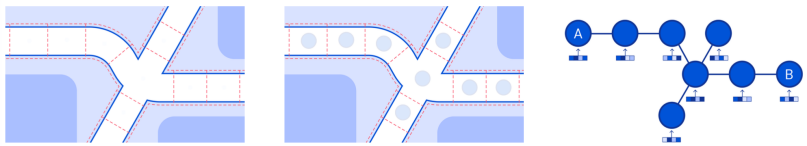


Multi-scale icosahedral mesh $M^0, M^1, \ldots, M^6$.

Google Maps: estimated time of arrival

Problem: predict travel time on road-navigation queries, at planet scale.

Graph: each road segment is a node; edges connect segments that meet at an intersection. Node features include live traffic, time-of-day, speed limit, road category.



Road segments → nodes → Supersegment graph.

GNN role: the global road graph is partitioned into overlapping *Supersegments*; a message-passing GNN predicts travel times at multiple horizons, and the per-Supersegment predictions are composed along the user's route.

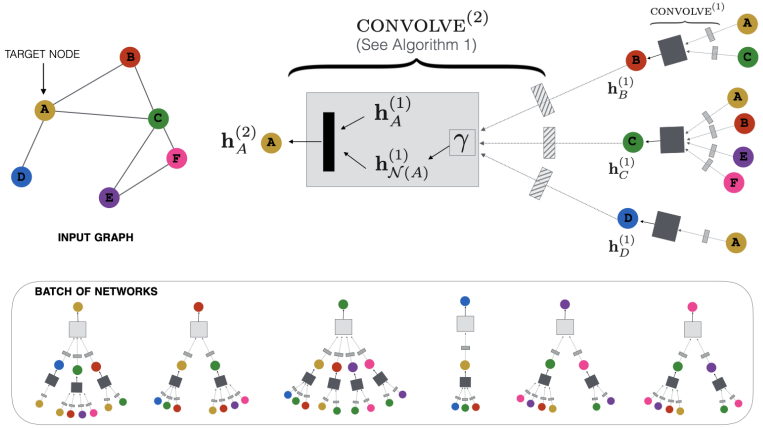
Headline: reduced negative-ETA outcomes by 40%+ in many cities—Sydney 43%, Osaka 37%, Orlando 34%, Singapore 31%, Taichung 51%. Google Maps serves >1B users/month.

(Derrow-Pinion et al., CIKM 2021.)

PinSage: GNNs for web-scale recommendation

Problem: given a user's interactions, recommend items they will find relevant. Interaction data forms a graph with hundreds of millions of nodes and billions of edges.

Graph: bipartite *pin–board* graph at Pinterest; vertices are pins and boards; edges connect a pin to every board on which it has been saved.



Two-layer aggregation over a target node's sampled neighborhood.

GNN role: computes a low-dimensional embedding for each pin by aggregating features from sampled local neighborhoods, using importance-sampled random walks to define a small, weighted receptive field.

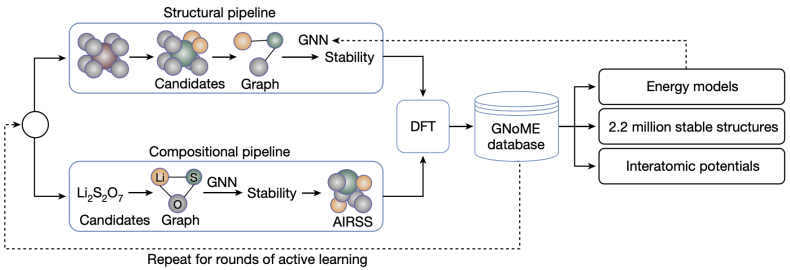
Headline: deployed on $\sim$3B nodes, $\sim$18B edges; preferred in $\sim$60% of head-to-head user studies. Powers the "Related Pins" feature at Pinterest.

(Ying et al., KDD 2018.)

GNoME: discovery of stable inorganic crystals

Problem: find new stable crystal structures for batteries, superconductors, semiconductors, solar cells.

Graph: atoms are vertices (features: element, local environment); edges connect atom pairs within a cutoff distance.



Structural + compositional pipelines, DFT verification, active learning.

GNN role: predicts formation energy—and hence thermodynamic stability—of a candidate crystal directly from its atomic graph. An active-learning loop uses density functional theory to verify top predictions and feeds verified structures back into the training set.

Headline: 2.2M new predicted stable structures; $\sim$381K on the convex hull of stability—an order-of-magnitude expansion, equivalent to $\sim$800 years of prior experimental discovery. External labs independently synthesized $>$700 of GNoME's predictions.

(Merchant et al., *Nature* 2023.)

Halicin: an antibiotic candidate identified by a GNN

Problem: identify new antibacterial compounds. (No new antibiotic class has been discovered since the 1980s.)

Graph: each molecule is a graph; atoms are vertices (element, charge, hybridization); chemical bonds are edges (order, stereo).

GNN role: a directed message-passing NN (D-MPNN) trained on 2,335 molecules screened against *E. coli* ranks a $\sim$6,000-compound repurposing library.

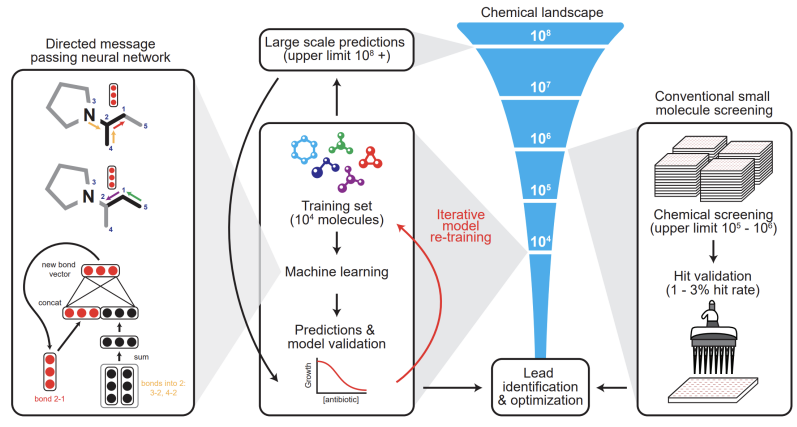


Figure 1. Machine Learning in Antibiotic Discovery
Modern approaches to antibiotic discovery often include screening large chemical libraries for those that elicit a phenotype of interest. These screens, which are upper bound by hundreds of thousands to a few million molecules, are expensive, time consuming, and can fail to capture an expansive breadth of chemical space. In contrast, machine learning approaches afford the opportunity to rapidly and inexpensively explore vast chemical spaces *in silico*. Our deep neural network model works by building a molecular representation based on a specific property, in our case the inhibition of the growth of *E. coli*, using a directed message passing approach. We first trained our neural network model using a collection of 2,335 diverse molecules for those that inhibited the growth of *E. coli*, augmenting the model with a set of molecular features, hyperparameter optimization, and ensembling. Next, we applied the model to multiple chemical libraries, comprising >107 million molecules, to identify potential lead compounds with activity against *E. coli*. After ranking the candidates according to the model's predicted score, we selected a list of promising candidates.

D-MPNN (left) + screening funnel (right) over 10^8 molecules.

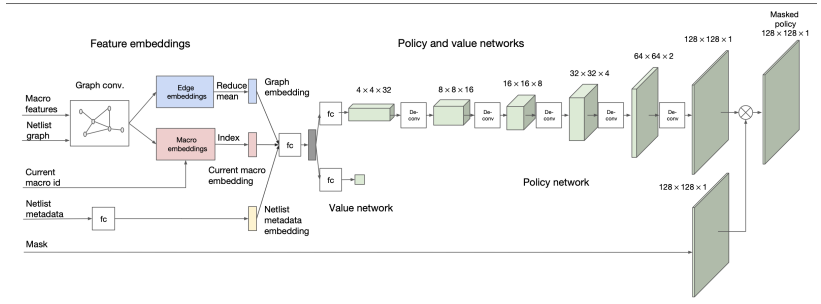
Headline: identified *halicin* (SU-3327), active against *C. difficile*, pan-resistant *A. baumannii*, and *M. tuberculosis*. Still preclinical—not yet in human trials.

(Stokes et al., *Cell* 2020.)

AlphaChip: AI chips designing AI chips

Problem: place *macros* on a 2D silicon canvas to minimize power, timing slack, area (PPA) and wirelength. Defied automation for 50+ years.

Graph: the *netlist*—a hypergraph whose vertices are macros and standard-cell clusters, and whose hyperedges are electrical nets.



Edge-GNN encoder (left) feeds policy + value heads (right).

GNN role: an edge-based GCN (*Edge-GNN*) encodes the partial placement state from the netlist hypergraph, feeding a reinforcement-learning policy that places one macro per step.

Headline: in under 6 hours, matches or beats the human physical-design team on all PPA metrics. Used in every Google TPU generation since v5e; adopted by MediaTek for Dimensity chips.

(Mirhoseini et al., *Nature* 2021.)

A common thread across the applications

Six very different domains:

atmospheric science, transportation, web-scale recommendation, materials chemistry, medicinal chemistry, hardware design.

What do they have in common?

- Not a common architecture, loss function, or dataset.
- But: in every case, the input is most faithfully represented as a **graph**.
- Treating it as such—rather than flattening it for an MLP or resampling it for a CNN—is what *unlocked the headline result*.

Every one of these systems is, at heart, a **message-passing computation** on a graph: each node aggregates information from its neighbors, applies a learned update, and passes a refined message on.

The rest of the course makes this statement precise.

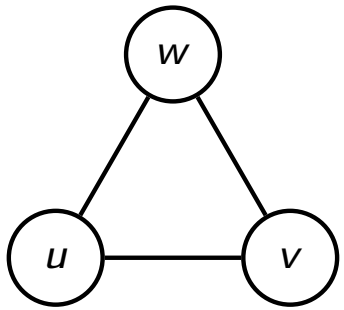
5 Basics of Graph Theory

Simple Graphs vs. Multigraphs

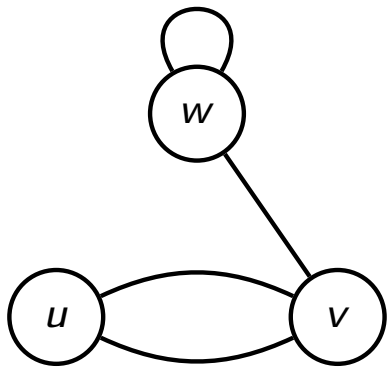
A simple graph has no self-loops and no parallel edges — at most one edge between any pair of vertices.

A multigraph may have **multiple (parallel) edges** connecting the same two vertices. If m edges connect u and v, we say $\{u, v\}$ has multiplicity m.

Simple graph



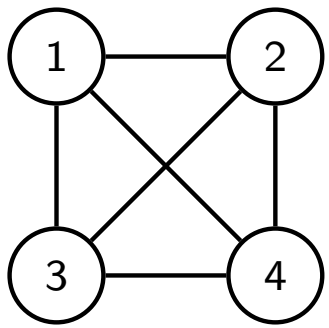
Multigraph



Special Graph Families: K_n and C_n

Complete graph K_n

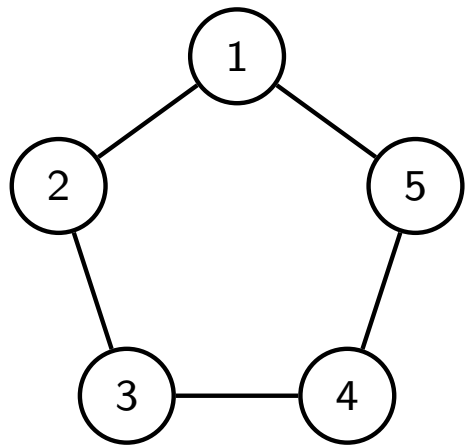
Every pair of vertices is connected by an edge. $|E| = \binom{n}{2}$.



K_4: 4 vertices, 6 edges

Cycle graph C_n

Vertices arranged in a single closed cycle. $|E| = n$.



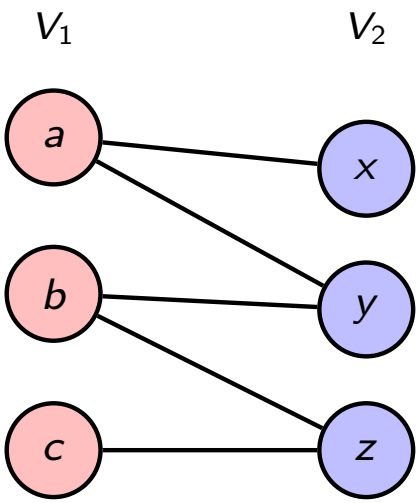
C_5: 5 vertices, 5 edges

Bipartite Graphs

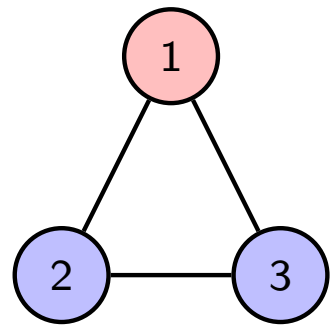
An undirected graph $G = (V, E)$ is bipartite if V can be partitioned into two disjoint subsets V_1 and V_2 such that every edge connects a vertex in V_1 to a vertex in V_2.

Equivalently: vertices can be 2-colored (red/blue) so that no two adjacent vertices share the same color.

Bipartite graph G ✓



Not bipartite H ×



odd cycle ⇒ not bipartite

Paths in a Graph

Definition: A path of length k from a vertex u to a vertex u' in $G = (V, E)$ is a sequence

$$p = \langle v_0, v_1, v_2, \ldots, v_k \rangle$$

of vertices such that $u = v_0$, $u' = v_k$, and $(v_{i-1}, v_i) \in E$ for $i = 1, 2, \ldots, k$.

- The length of a path is the number of edges in it (i.e., k).
- We say u' is reachable from u via p if such a path p exists.
- A path is simple if all vertices $v_0, v_1, \ldots, v_k$ are distinct.

Neighborhood of a Node

Let $G = (V, E)$ be a graph.

Definition: The neighborhood of a node $v \in V$, denoted $N(v)$, is the set of neighbors of v in G:

$$N(v) := \{u \in V : \{u, v\} \in E\}.$$

Definition: The distance in G between nodes u and v, denoted $d_G(u, v)$, is the length of a shortest path in G from u to v. If no such path exists, $d_G(u, v) = \infty$.

Definition: The K-hop neighborhood ($K \geq 1$) of a node $v \in V$ is the set

$$N_K(v) := \{u \in V : d_G(u, v) \leq K\}.$$

Remark (GNN connection):

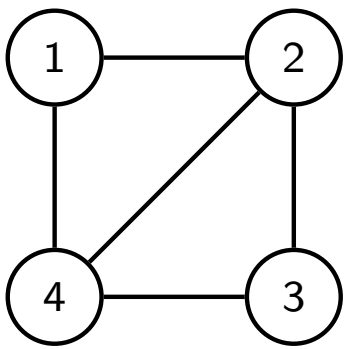
- A K-layer GNN computes the representation of each node v by aggregating information from v's K-hop neighborhood $N_K(v)$.
- The depth of the GNN network therefore controls how far each node "sees" into the graph.

Adjacency Matrix Representation of a Graph

Definition: Let $G = (V, E)$ be a graph with $V = \{v_1, \ldots, v_n\}$. The adjacency matrix of G is the $n \times n$ matrix $\mathbf{A}$ defined by

$$\mathbf{A}[i,j] = \begin{cases} 1 & \text{if } \{v_i, v_j\} \in E, \\ 0 & \text{otherwise.} \end{cases}$$

For an undirected graph, $\mathbf{A}$ is symmetric: $\mathbf{A}[i,j] = \mathbf{A}[j,i]$.

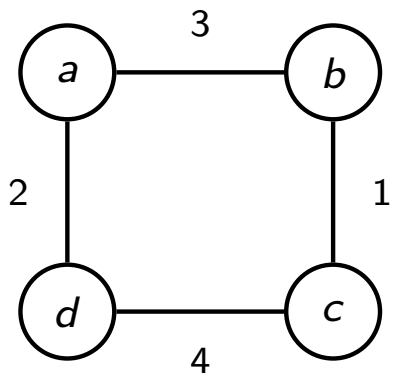


$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

Weighted Graphs

Definition: A weighted graph is a triple $G = (V, E, w)$, where (V, E) is a graph and $w : E \to \mathbb{R}$ assigns a weight to each edge.

The adjacency matrix generalizes: $\mathbf{A}[i, j] = w(\{v_i, v_j\})$ if $\{v_i, v_j\} \in E$, and 0 otherwise.



$$\mathbf{A} = \begin{pmatrix} 0 & 3 & 0 & 2 \\ 3 & 0 & 1 & 0 \\ 0 & 1 & 0 & 4 \\ 2 & 0 & 4 & 0 \end{pmatrix}$$

Example: In a road network, the weight of an edge can represent the distance or travel time along that road segment.

Node Degree

Definition: Let $G = (V, E)$ be a graph with adjacency matrix $\mathbf{A}$. The degree of a node $u \in V$ is

$$d_u \;=\; |N(u)| \;=\; \sum_{v \in V} \mathbf{A}[u, v].$$

For a **directed** graph, one distinguishes:

- In-degree: $d_u^{\text{in}} = \sum_{v \in V} \mathbf{A}[v, u]$ (number of edges *into* u).
- Out-degree: $d_u^{\text{out}} = \sum_{v \in V} \mathbf{A}[u, v]$ (number of edges *out of* u).

Theorem (Handshaking Lemma): In any undirected graph $G = (V, E)$,

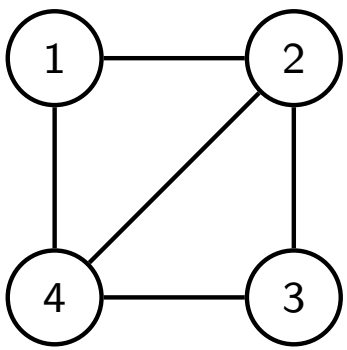
$$\sum_{v \in V} d_v \;=\; 2\,|E|.$$

Degree Matrix

Definition: The degree matrix of a graph $G = (V, E)$ with $n = |V|$ is the $n \times n$ diagonal matrix $\mathbf{D}$ defined by

$$\mathbf{D}[i, j] = \begin{cases} d_{v_i} & \text{if } i = j, \\ 0 & \text{otherwise,} \end{cases}$$

where d_{v_i} is the degree of vertex v_i.



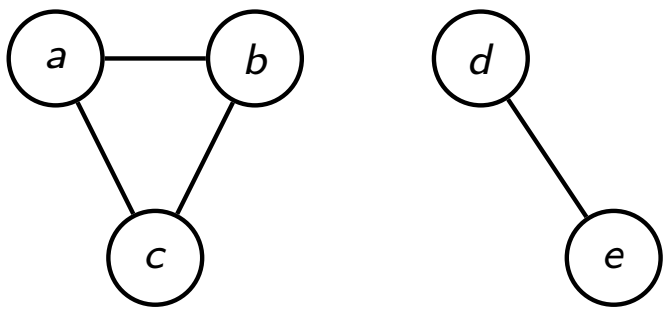
$$\mathbf{D} = \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 3 \end{pmatrix}$$

The degree matrix appears in many GNN constructions, e.g. in the definition of the graph Laplacian and in degree-based normalization of adjacency matrices.

Connected Graphs and Connected Components

Definition: An undirected graph $G = (V, E)$ is connected if for every pair of vertices $u, v \in V$ there exists a path from u to v in G.

Definition: A connected component of G is a maximal connected subgraph of G.



This graph has **two** connected components: $\{a, b, c\}$ and $\{d, e\}$.

$d_G(a, d) = \infty$ since no path connects them.

Remark: Most GNN architectures operate independently on each connected component, since message passing cannot cross between disconnected parts of the graph.

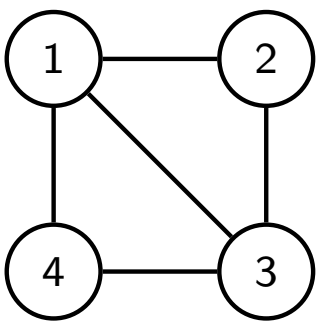
Subgraphs

Definition: A graph $H = (V', E')$ is a subgraph of $G = (V, E)$, written $H \subseteq G$, if $V' \subseteq V$ and $E' \subseteq E$.

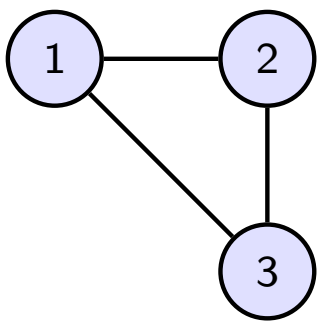
Definition: The subgraph of G induced by a vertex set $S \subseteq V$ is the graph $G[S] = (S, E_S)$, where

$$E_S = \big\{\{u, v\} \in E : u \in S \text{ and } v \in S\big\}.$$

Graph G



$G[\{1, 2, 3\}]$



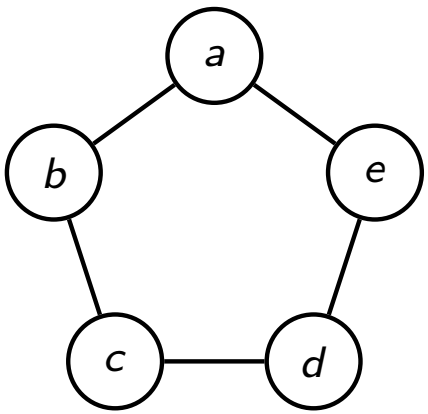
Remark: In GNNs, the notion of a node's *ego graph* (the induced subgraph on $\{v\} \cup N(v)$) and its K-hop induced subgraph are central to understanding the receptive field of each node.

Graph Isomorphism

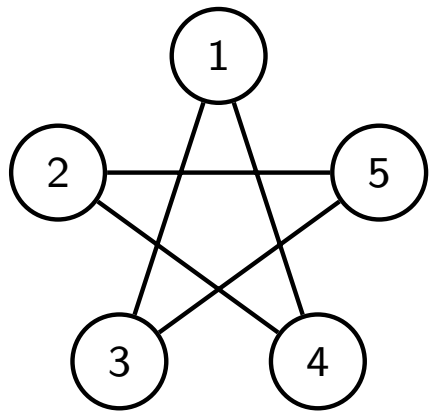
Definition: Two graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are isomorphic, written $G_1 \cong G_2$, if there exists a bijection $\phi : V_1 \to V_2$ such that

$$\{u, v\} \in E_1 \iff \{\phi(u), \phi(v)\} \in E_2.$$

G_1 (pentagon)



G_2 (pentagram)



Both are C_5 — same structure, different drawings. The bijection $\phi(a) = 1,\ \phi(b) = 3,\ \phi(c) = 5,\ \phi(d) = 2,\ \phi(e) = 4$ preserves edges and non-edges.

Why it matters for GNNs: A maximally powerful GNN should map isomorphic graphs to the same representation and non-isomorphic graphs to different representations. The *Weisfeiler–Lehman (WL) test* provides an upper bound on the discriminative power of message-passing GNNs (Xu et al., 2019).

Node and Edge Features (Attributed Graphs)

In many real-world graphs, each node and/or edge carries additional **feature** (attribute) information beyond the graph topology.

Definition: An attributed graph is a tuple $G = (V, E, \mathbf{X})$, where $\mathbf{X} \in \mathbb{R}^{|V| \times d}$ is the node feature matrix: row $\mathbf{x}_v \in \mathbb{R}^d$ is the feature vector of node v.

Domain	Node features	Edge features
Social network	User profile, age	Interaction frequency
Molecule	Atom type, charge	Bond type, length
Citation graph	Paper text (bag-of-words)	—
Traffic network	Sensor readings	Road segment length

Remark: GNNs take $(\mathbf{A}, \mathbf{X})$ as input. If no natural features exist, common choices are one-hot node identities, node degree, or constant vectors.

Multi-relational Graphs

Definition: A multi-relational graph is a graph in which edges carry *types* (relations). An edge is a triple (u, τ, v), where $\tau \in \mathcal{R}$ is the relation type. For each relation τ there is an adjacency matrix $\mathbf{A}_\tau \in \{0,1\}^{|V| \times |V|}$.

Two important special cases:

- **Heterogeneous graph:** nodes also have types, i.e. $V = V_1 \cup V_2 \cup \cdots \cup V_k$ (disjoint), and certain edge types connect only specific node types.

  Example: A biomedical graph with node types {protein, drug, disease} and edge types {treats, interacts, causes}.

- **Multiplex graph:** every node belongs to every *layer*; each layer has its own edge type (intra-layer edges), plus inter-layer edges connecting the same node across layers.

  Example: A transportation network where each layer is a different mode (air, rail, road).

The Graph Laplacian

Definition: Let G be a graph with adjacency matrix $\mathbf{A}$ and degree matrix $\mathbf{D}$. The (unnormalized) graph Laplacian is

$$\mathbf{L} = \mathbf{D} - \mathbf{A}.$$

Key properties:

- $\mathbf{L}$ is symmetric and positive semi-definite.
- For any $\mathbf{x} \in \mathbb{R}^{|V|}$: $\mathbf{x}^\top \mathbf{L}\, \mathbf{x} = \sum_{\{u,v\} \in E} \left(x_u - x_v\right)^2$.
- The multiplicity of eigenvalue 0 equals the number of connected components of G.

The normalized Laplacian is $\mathbf{L}_{\text{sym}} = \mathbf{D}^{-1/2}\, \mathbf{L}\, \mathbf{D}^{-1/2} = \mathbf{I} - \mathbf{D}^{-1/2}\, \mathbf{A}\, \mathbf{D}^{-1/2}$.

Why it matters: Spectral GNNs (e.g. ChebNet, GCN) define graph convolutions via the eigendecomposition of $\mathbf{L}$ or $\mathbf{L}_{\text{sym}}$.

Acknowledgements

Image sources are credited on individual slides where applicable.